

NBACore v8.3.13

Configuration System

AI Development Work Contract

Version

v8.3.13

Module Type

Core Infrastructure

Execution Target

WorkBuddy / Codex / Local AI Coding Agent

1. Module Objective

建立 NBACore 全局配置管理系统。

目标：

将系统行为从代码中分离。

实现：

Configuration

|

System Behavior

|

Modules

2. Problem Definition

随着 NBACore 发展：

系统会出现大量可变因素。

例如：

联赛

NBA:

Quarter = 12 minutes

FIBA:

Quarter = 10 minutes

用户环境

中文：

kg

米

英文：

lbs

feet

数据源

不同 API：

NBA API

Kaggle Dataset

User Excel

如果全部写死：

```
if league=="NBA":  
    do_xxx()
```

项目会快速失控。

3. Core Principle

所有可变化内容：

进入配置层。

代码负责：

How

配置负责：

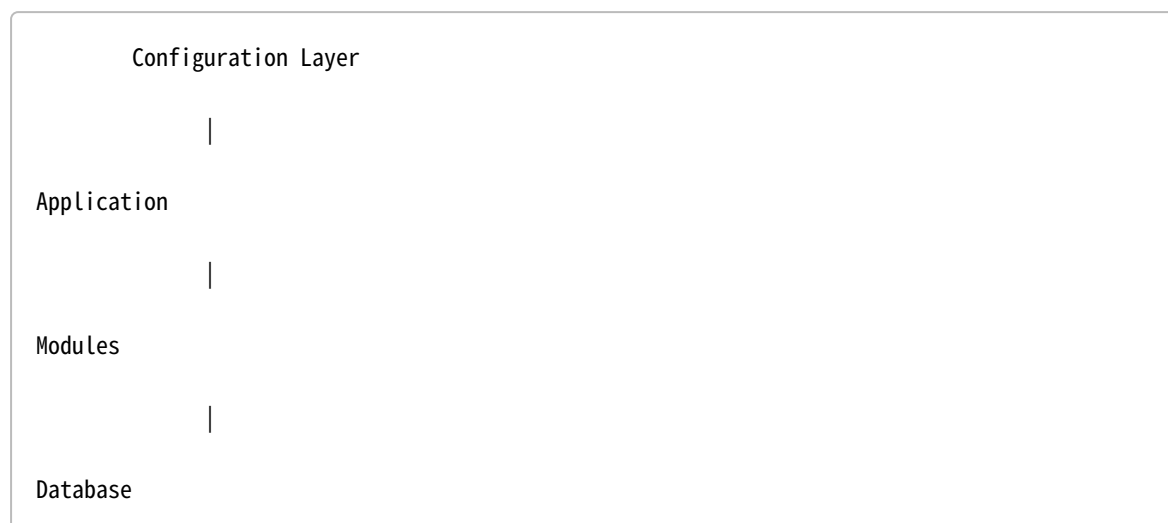
What

4. Architecture Position

当前：

```
Application  
|  
Modules  
|  
Database
```

加入：



5. Configuration Categories

系统配置分为：

5.1 System Configuration

控制基础运行。

例如：

```
{
  "environment": "production",
  "debug": false
}
```

5.2 User Configuration

用户偏好。

例如：

```
{
  "language": "zh-CN",
```

```
"theme":"dark"  
}
```

5.3 Competition Configuration

联赛配置。

例如：

```
{  
  "league":"NBA",  
  
  "ruleset":"nba_2026"  
}
```

5.4 Data Configuration

数据相关。

例如：

```
{  
  "default_source":"csv",  
  
  "auto_mapping":true  
}
```

5.5 Visualization Configuration

图表设置。

例如：

```
{  
  "default_chart":"bar",  
  
  "show_percentage":true  
}
```

5.6 Plugin Configuration

插件管理。

例如：

```
{
  "enabled_plugins":[
    "nba_adapter",
    "fiba_adapter"
  ]
}
```

6. Data Model

6.1 Configuration Table

```
CREATE TABLE configurations(
  id INTEGER PRIMARY KEY,
  category TEXT,
  key TEXT,
  value JSON,
  owner_id INTEGER,
  created_at DATETIME,
  updated_at DATETIME
);
```

7. Configuration File Structure

推荐：

```
config/  
  
  system.json  
  
  user.json  
  
  competition.json  
  
  plugins.json  
  
  visualization.json
```

8. Example Configuration

system.json

```
{  
  "version":"8.3.13",  
  
  "debug":true,  
  
  "database":"sqlite"  
}
```

competition.json

```
{  
  
  "default":"NBA",  
  
  "available":[  
  
    "NBA",  
  
    "FIBA",  
  
    "NCAA"  
  ]  
}
```

9. Backend Structure

创建：

```
backend/  
  
services/  
  
configuration/  
    manager.py  
    loader.py  
    validator.py
```

10. Core Classes

ConfigurationManager

```
class ConfigurationManager:  
  
    def get(self, key):  
        pass  
  
    def set(self, key, value):  
        pass  
  
    def reset(self, key):  
        pass
```

ConfigurationValidator

负责：

检查配置合法性。

例如：

错误：

```
{  
  "quarter": "abc"  
}
```

应该拒绝。

11. API Requirements

Get Configuration

GET:

```
/api/config/{category}
```

返回：

```
{  
  
  "language": "zh-CN",  
  
  "unit": "metric"  
}
```

Update Configuration

POST:

```
/api/config
```

Request:

```
{  
  
  "category": "user",  
  
  "key": "language",  
}
```

```
"value": "en-US"
}
```

12. Integration Requirements

必须接入：

Ontology Layer

控制：

字段语言。

例如：

中文字段显示。

Rules Engine

控制：

联赛规则。

Plugin System

控制：

插件启停。

Visualization

控制：

图表默认行为。

13. Frontend Requirements

第一阶段：

提供：

Settings Interface

包括：

Language

中文

English

Units

Metric

Imperial

Competition

NBA

FIBA

NCAA

Plugins

Enable

Disable

14. Testing Requirements

Unit Test

测试：

读取配置

修改配置

恢复默认

Integration Test

测试：

修改：

League = FIBA

系统：

自动加载 FIBA ruleset。

15. Acceptance Criteria

完成标准：

系统可以：

- ✓ 保存配置
 - ✓ 动态读取配置
 - ✓ 不修改代码切换环境
 - ✓ 支持用户配置
 - ✓ 支持插件配置
-

16. Development Notes For AI Agent

重要：

禁止：

大量使用硬编码。

错误：

```
NBA_QUARTER=12
```

正确：

```
config.rules.quarter_length
```

所有未来模块：

必须优先读取 Configuration Layer。

17. Future Extension

未来支持：

AI Configuration

控制：

AI模型参数。

Analysis Profile

保存：

用户分析习惯。

Team Profile

保存：

球队分析模板。

Final Principle

NBACore 不应该是一个固定软件。

应该成为：

Configurable Basketball Intelligence Framework

配置系统是未来：

- 多联赛
- 插件生态
- AI分析
- 全球化

的基础。

END